TASK 1

AIM:To check whether the two halves of a given string contain an equal number of vowels.

ALGORITHM

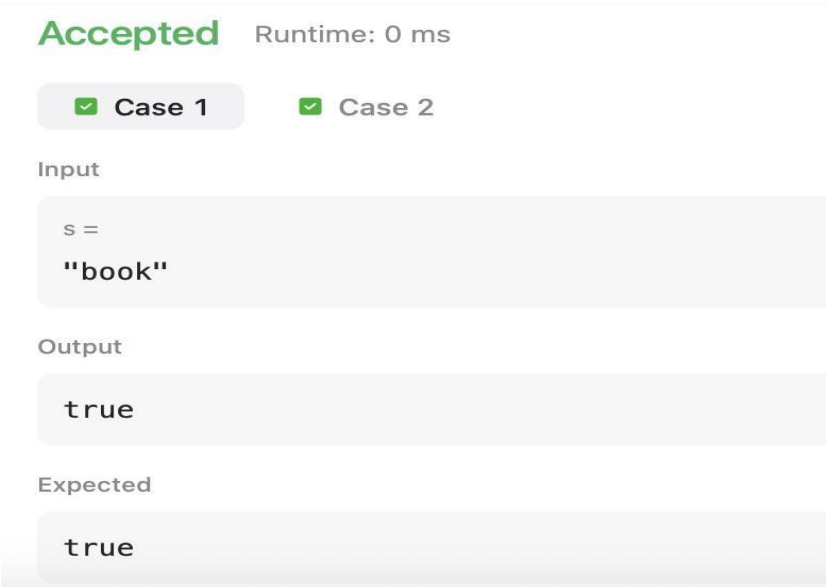
1. Read the input string s.
2. Find the length of the string and divide it into two equal halves.
3. Initialize two counters:
   - countA for vowels in the first half
   - countB for vowels in the second half
4. Traverse the first half of the string:
   - If a character is a vowel, increment countA.
5. Simultaneously check the corresponding character in the second half:
   - If it is a vowel, increment countB.
6. Compare countA and countB.
7. If both counts are equal, return true; otherwise, return false.

PROGRAM

```
import java.util.Scanner;

class Solution {

    public boolean halvesAreAlike(String s) {
        int n = s.length();
        int mid = n / 2;

        int countA = 0;
        int countB = 0;

        for (int i = 0; i < mid; i++) {
            if (isVowel(s.charAt(i))) {
                countA++;
            }
            if (isVowel(s.charAt(i + mid))) {
                countB++;
            }
        }

        return countA == countB;
    }

    private boolean isVowel(char ch) {
        ch = Character.toLowerCase(ch);
        return ch == 'a' || ch == 'e' || ch == 'i' ||
               ch == 'o' || ch == 'u';
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.println("Enter a string:");
        String s = sc.nextLine();

        Solution obj = new Solution();
```

```
        boolean result = obj.halvesAreAlike(s);

        if (result) {
            System.out.println("Both halves have equal number of vowels.");
        } else {
            System.out.println("Both halves do not have equal number of vowels.");
        }

        sc.close();
    }
}
```

OUTPUT



RESULT :The program successfully determines whether both halves of the string contain the same number of vowels.

TASK 2

AIM:To check whether a given string is a Lapindrome.

ALGORITHM:

1. Read the input string s.
2. Find the length of the string n.
3. Create two integer arrays freq1 and freq2 of size 26 to store character frequencies.
4. Calculate the middle index mid = n / 2.
5. Store the frequency of characters from index 0 to mid - 1 in freq1.
6. If the string length is odd, skip the middle character.
7. Store the frequency of characters from the second half of the string in freq2.
8. Compare both frequency arrays:
   - If all frequencies match, the string is a Lapindrome.
   - Otherwise, it is not a Lapindrome.
9. Print "YES" if the string is a Lapindrome, else print "NO".

PROGRAM

```
import java.util.*;
import java.lang.*;
import java.io.*;
class Codechef {
   public static void main (String[] args) throws java.lang.Exception {
      Scanner sc = new Scanner(System.in);
      int T = sc.nextInt();
      while (T-- > 0) {
         String s = sc.next();
         int n = s.length();
         int[] freq1 = new int[26];
         int[] freq2 = new int[26];
         int mid = n / 2;

         for (int i = 0; i < mid; i++) {
            freq1[s.charAt(i) - 'a']++;
         }
         for (int i = (n % 2 == 0 ? mid : mid + 1); i < n; i++) {
            freq2[s.charAt(i) - 'a']++;
        }
         boolean isLapindrome = true;
         for (int i = 0; i < 26; i++) {
            if (freq1[i] != freq2[i]) {
               isLapindrome = false;
               break;
            }
         }
        if (isLapindrome)
            System.out.println("YES");
         else
            System.out.println("NO");
      }
      sc.close();
   }
}
```

OUTPUT:

Input

```
3
gaga
abcba
abca
```

Output

```
objectivec

YES
YES
NO
```

RESULT:The program successfully determines whether each given string is a Lapindrome.

TASK 3

AIM: To compare the ratings of Alice and Bob in three categories and calculate their comparison scores based on given rules.

ALGORITHM:

1. Start
2. Read three integers into list a (Alice's ratings).
3. Read three integers into list b (Bob's ratings).
4. Initialize two variables:
   - alice = 0
   - bob = 0
5. For i from 0 to 2:
   - If a[i] > b[i], increment alice
   - Else if a[i] < b[i], increment bob
   - Else, do nothing
6. Store alice and bob scores in a result list.
7. Print the result.
8. Stop

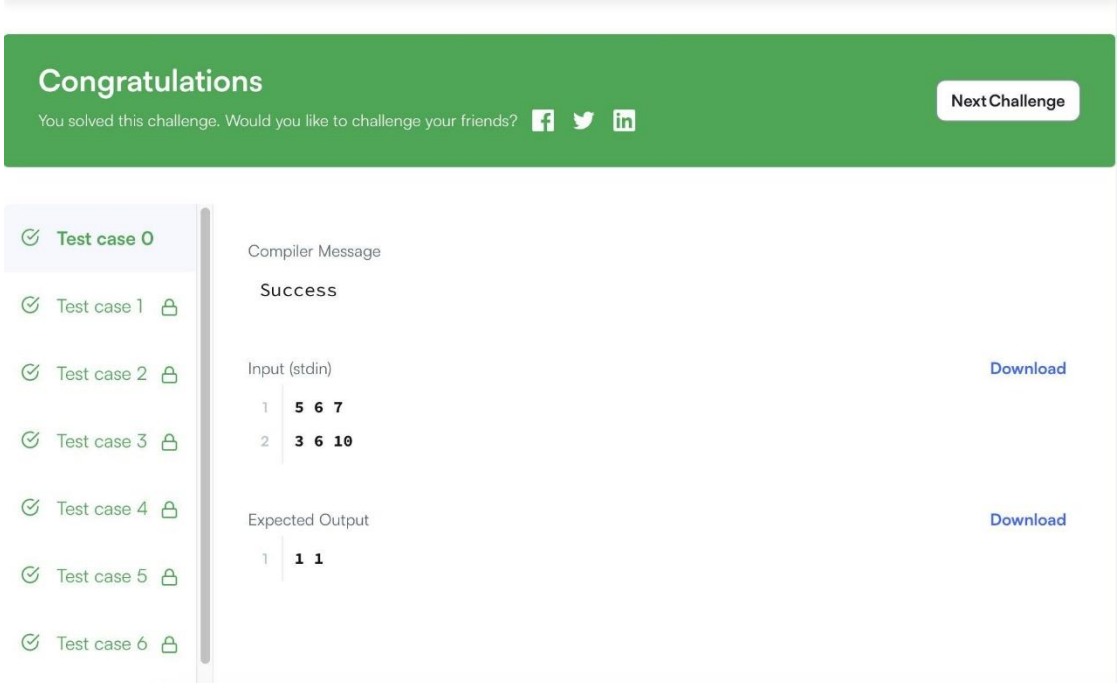
PROGRAM:

```
import java.io.*; import java.util.*;

class Result {


public static List<Integer> compareTriplets(List<Integer> a, List<Integer> b) { int alice = 0;
int bob = 0;

for (int i = 0; i < 3; i++) { if (a.get(i) > b.get(i)) {
alice++;
} else if (a.get(i) < b.get(i)) { bob++;
}
}

List<Integer> result = new ArrayList<>(); result.add(alice);
result.add(bob);

return result;
}
}
```

```
for (int i = 0; i < 3; i++) { a.add(Integer.parseInt(aTemp[i]));
}

String[] bTemp = bufferedReader.readLine().trim().split(" "); List<Integer> b = new ArrayList<>();

for (int i = 0; i < 3; i++) { b.add(Integer.parseInt(bTemp[i]));
}

List<Integer> result = Result.compareTriplets(a, b);

bufferedWriter.write(result.get(0) + " " + result.get(1)); bufferedWriter.newLine();

bufferedReader.close(); bufferedWriter.close();
}
}import java.io.*; import java.util.*;

class Result {


public static List<Integer> compareTriplets(List<Integer> a, List<Integer> b) { int alice = 0;
int bob = 0;

for (int i = 0; i < 3; i++) { if (a.get(i) > b.get(i)) {
alice++;
} else if (a.get(i) < b.get(i)) { bob++;
}
}

List<Integer> result = new ArrayList<>(); result.add(alice);
result.add(bob);

return result;
}
}
```

```
for (int i = 0; i < 3; i++) { a.add(Integer.parseInt(aTemp[i]));
}

String[] bTemp = bufferedReader.readLine().trim().split(" "); List<Integer> b = new ArrayList<>();

for (int i = 0; i < 3; i++) { b.add(Integer.parseInt(bTemp[i]));
}

List<Integer> result = Result.compareTriplets(a, b);

bufferedWriter.write(result.get(0) + " " + result.get(1)); bufferedWriter.newLine();

bufferedReader.close(); bufferedWriter.close();
}
}.
```

OUTPUT:



RESULT:The program successfully compares Alice’s and Bob’s ratings and prints their final scores (Alice first, Bob second)

TASK 4

AIM:To check whether an integer array contains any duplicate elements. ALGORITHM:

1. Create an empty HashSet.
2. Traverse each element in the array.
3. If the element already exists in the set, return true.
4. Otherwise, add the element to the set.
5. If the loop ends without finding duplicates, return false.

PROGRAM:

```
import java.util.*; class Solution {
public boolean containsDuplicate(int[] nums) { Set<Integer> seen = new HashSet<>();
for (int num : nums) {
if (seen.contains(num)) { return true;
}
seen.add(num);
}

return false;
}
}
```

OUTPUT:

RESULT:The program successfully detects duplicates in the array and returns true if any element appears more than once; otherwise, it returns false.

TASK 5

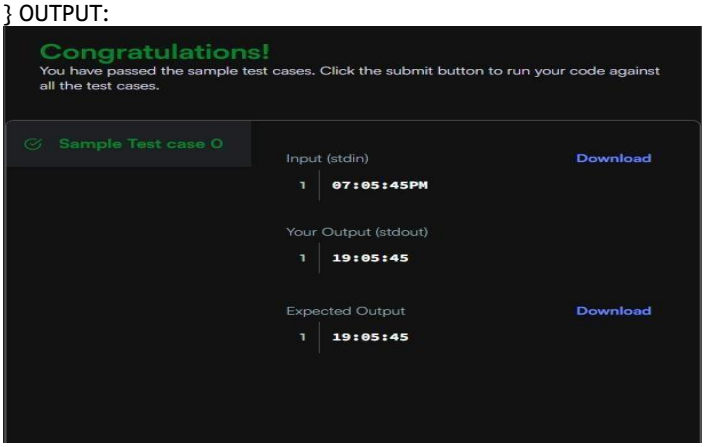
AIM: To convert a given time from **12-hour AM/PM format** into **24-hour (military) format** using Java. ALGORITHM:

1. Read the input time string in the format hh:mm:ssAM or hh:mm:ssPM.
2. Extract:
    - The last two characters to determine AM or PM.
    - The first two characters as the hour.
    - The remaining part :mm:ss.
3. If the time is **AM**:
    - Convert 12 to 00.
4. If the time is **PM**:
    - Add 12 to the hour if it is not already 12.
5. Format the hour to always have two digits.
6. Append the minutes and seconds .
7. Return the converted time in **24-hour format**.

PROGRAM :

```
import java.io.*; class Result {
public static String timeConversion(String s) {

String period = s.substring(s.length() - 2); // AM or PM int hour = Integer.parseInt(s.substring(0, 2));
String rest = s.substring(2, 8); // :mm:ss if (period.equals("AM")) {
if (hour == 12) { hour = 0;
}

} else { // PM

if (hour != 12) { hour += 12;}}
```

```
return String.format("%02d%s", hour, rest);

}}

public class Solution {

public static void main(String[] args) throws IOException { BufferedReader bufferedReader = new BufferedReader(
new InputStreamReader(System.in)

);

BufferedWriter bufferedWriter = new BufferedWriter( new FileWriter(System.getenv("OUTPUT_PATH"))
);

String s = bufferedReader.readLine(); String result = Result.timeConversion(s); bufferedWriter.write(result);
bufferedWriter.newLine(); bufferedReader.close(); bufferedWriter.close();}
} OUTPUT:
```



RESULT: The program successfully converts the given 12-hour format time into its corresponding **24-hour format**.

TASK 6

AIM: To move all zero elements in an array to the end while maintaining the relative order of the non-zero elements.

ALGORITHM;

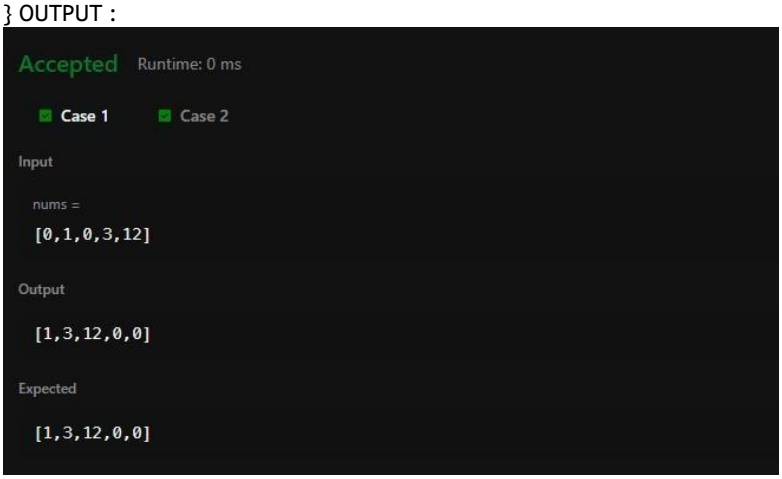
1. Initialize an index variable to 0.
2. Traverse the array from left to right.
3. Whenever a non-zero element is found, place it at the position indicated by index and increment index.
4. After all non-zero elements are moved forward, fill the remaining positions in the array with 0.
5. Display the modified array.

PROGRAM

```
import java.util.Scanner; class Solution {
public void moveZeroes(int[] nums) { int index = 0;
for (int i = 0; i < nums.length; i++) { if (nums[i] != 0) {
nums[index] = nums[i]; index++;
}

}

while (index < nums.length) { nums[index] = 0; index++;}
public static void main(String[] args) { Scanner sc = new Scanner(System.in);
System.out.print("Enter number of elements: "); int n = sc.nextInt();
int[] nums = new int[n]; System.out.println("Enter elements:"); for (int i = 0; i < n; i++) {
```

```
nums[i] = sc.nextInt();

}Solution sol = new Solution(); sol.moveZeroes(nums); System.out.print("Array after moving zeros: "); for (int num :
nums) {
System.out.print(num + " ");

}

sc.close();

}
```

} OUTPUT :



RESULT: The program rearranges the array by moving all zeros to the end without changing the order of non-zero elements.

TASK 7

AIM: To calculate the **absolute difference between the sums of the primary and secondary diagonals** of a given square matrix.

ALGORITHM:

1. Read the integer n, representing the size of the square matrix.
2. Read the n × n elements of the matrix into a 2-D list.
3. Initialize two variables:

- primarySum for the left-to-right diagonal.
- secondarySum for the right-to-left diagonal.

4. Traverse the matrix using a loop:

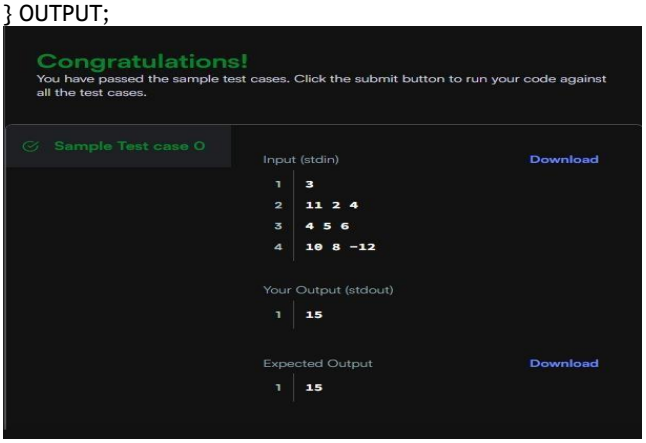
- Add arr[i][i] to primarySum.
- Add arr[i][n - 1 - i] to secondarySum.

5. Compute the absolute difference between the two sums.
6. Display the result.

PROGRAM

```
import java.io.*; import java.util.*; class Result {
public static int diagonalDifference(List<List<Integer>> arr) { int n = arr.size();
int primarySum = 0; int secondarySum = 0;
for (int i = 0; i < n; i++) { primarySum += arr.get(i).get(i);
secondarySum += arr.get(i).get(n - 1 - i);} return Math.abs(primarySum - secondarySum);
}

public class Solution {

public static void main(String[] args) throws IOException { BufferedReader bufferedReader = new BufferedReader(
new InputStreamReader(System.in)
```

```
);

BufferedWriter bufferedWriter = new BufferedWriter( new FileWriter(System.getenv("OUTPUT_PATH"))
); int n = Integer.parseInt(bufferedReader.readLine().trim()); List<List<Integer>> arr = new ArrayList<>();
for (int i = 0; i < n; i++) {

String[] arrRowTempItems = bufferedReader.readLine().trim().split(" "); List<Integer> arrRowItems = new
ArrayList<>();
for (int j = 0; j < n; j++) { arrRowItems.add(Integer.parseInt(arrRowTempItems[j]));}
arr.add(arrRowItems);

} int result = Result.diagonalDifference(arr); bufferedWriter.write(String.valueOf(result));
bufferedWriter.newLine(); bufferedReader.close(); bufferedWriter.close(); }
} OUTPUT;
```



RESULT: The program successfully computes and displays the **absolute difference between the sums of both diagonals** of the given square matrix.

TASK 8
AIM: The goal is to transform a given 2D integer array (matrix) by swapping its rows and columns. Specifically, the element at position matrix[i][j] should move to result[j][i] in the new matrix.

ALGORITHM:

1. **Identify Dimensions**: Find the number of rows (m) and columns (n) of the original matrix.
2. **Initialize Result**: Create a new 2D array result with dimensions $n \times m$ (the reverse of the original).
3. **Nested Iteration**:
4. Outer loop: Iterate through each row i from 0 to $m-1$.
5. Inner loop: Iterate through each column j from 0 to $n-1$.
6. **Value Assignment**: Map the value: result[j][i] = matrix[i][j].
7. **Return**: Once all elements are processed, return the result matrix

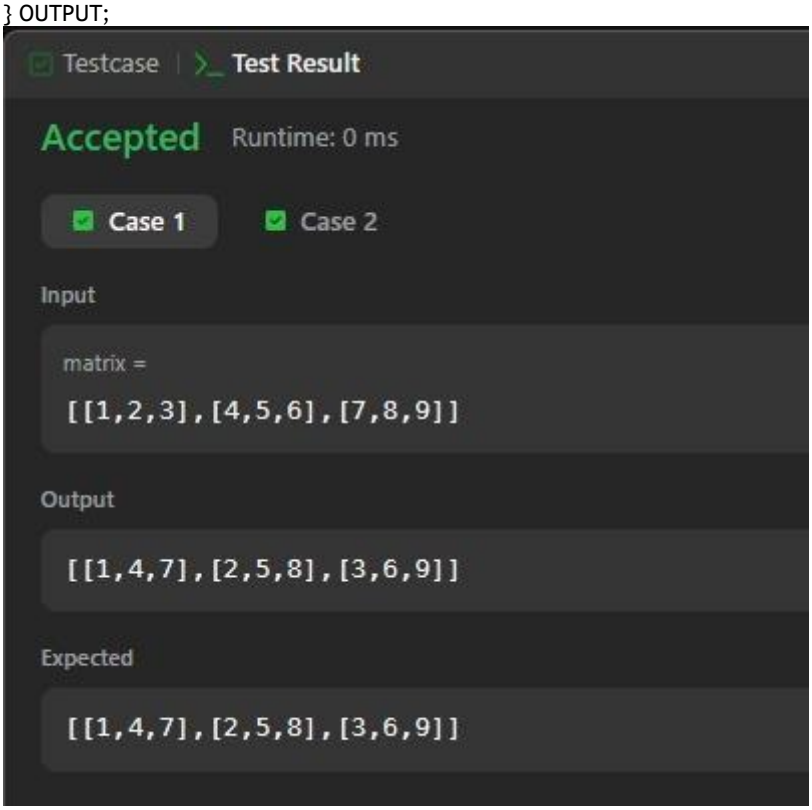
PROGRAM:

```
import java.util.Arrays; class Solution {
public int[][] transpose(int[][] matrix) { int m = matrix.length;
int n = matrix[0].length; int[][] result = new int[n][m] for (int i = 0; i < m; i++) {
for (int j = 0; j < n; j++) result[j][i] = matrix[i][j] ;}
}

return result;

public static void main(String[] args) {

Solution sol = new Solution int[][] matrix = {
{1, 2, 3},

{4, 5, 6}
```

}

int[][] output = sol.transpose(matrix); System.out.println("Transposed Matrix:"); for (int[] row : output) { System.out.println(Arrays.toString(row));}}

} OUTPUT;



RESULT ; The program has been executed successfully, and the resulting transposed matrix is generated as a new two-dimensional array where the original rows have been converted into columns.

TASK 9

AIM: To compute the block sum of each element in a given 2D matrix using an efficient prefix sum technique.

ALGORITHM:

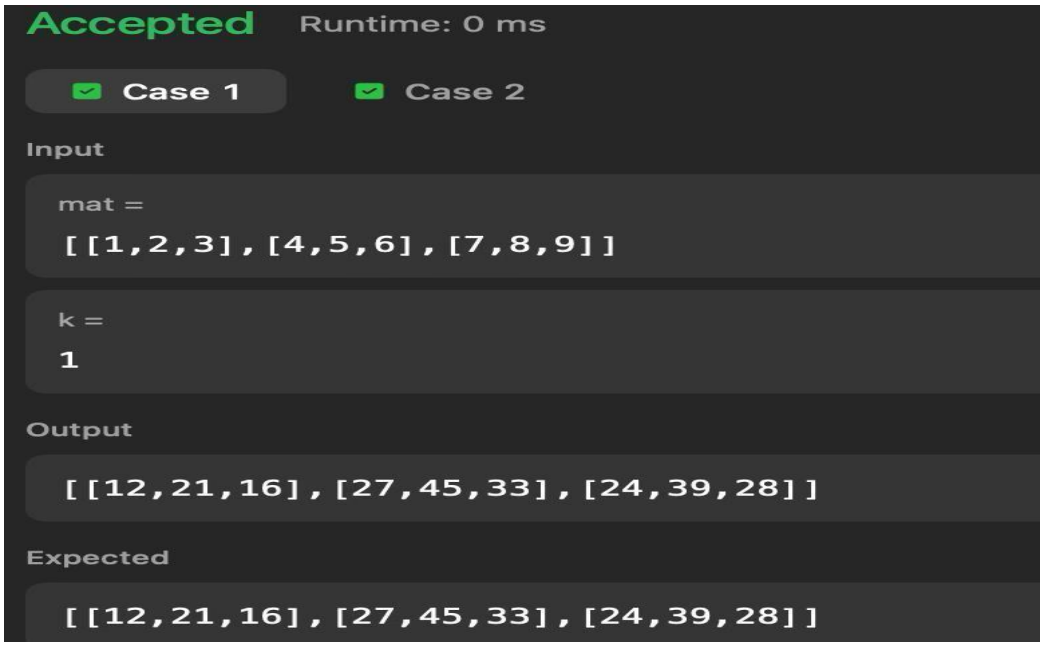
1. Read the matrix mat and integer k.
2. Get dimensions m (rows) and n (columns).
3. Create a prefix sum matrix of size (m+1) x (n+1).
4. Fill the prefix matrix using:
   - prefix[i][j] =
   - mat[i-1][j-1]
   - prefix[i-1][j]
   - prefix[i][j-1]
   - prefix[i-1][j-1]
5. Create result matrix answer[m][n].
6. For each element (i, j):
7. Calculate block boundaries:
   - r1 = max(0, i-k)
   - c1 = max(0, j-k)
   - r2 = min(m-1, i+k)
   - c2 = min(n-1, j+k)
8. Convert to prefix indexing.
9. Compute block sum using prefix formula.
10. Store the result in answer[i][j].
11. Return the result matrix.

PROGRAM:

```
class Solution {
public int[][] matrixBlockSum(int[][] mat, int k) { int m = mat.length;
int n = mat[0].length;
int[][] prefix = new int[m + 1][n + 1]; for (int i = 1; i <= m; i++) {
for (int j = 1; j <= n; j++) {
prefix[i][j] = mat[i - 1][j - 1]
+ prefix[i - 1][j]
+ prefix[i][j - 1]
- prefix[i - 1][j - 1];
}
}
int[][] answer = new int[m][n]; for (int i = 0; i < m; i++) {
for (int j = 0; j < n; j++) {

int r1 = Math.max(0, i - k); int c1 = Math.max(0, j - k);
```

```
int r2 = Math.min(m - 1, i + k); int c2 = Math.min(n - 1, j + k);
r1++; c1++; r2++; c2++;
answer[i][j] = prefix[r2][c2]
                              - prefix[r1 - 1][c2]
                              - prefix[r2][c1 - 1]
+ prefix[r1 - 1][c1 - 1];
}
}
return answer;
}
}
```

OUTPUT



RESULT : The program successfully computes the sum of all elements within a k distance block for each cell in the matrix using an optimized O(m × n) approach.

TASK 10

AIM:To rotate a given 2D matrix r times in the anti-clockwise direction by rotating each layer (ring) independently.

ALGORITHM:

1. Read the values of:
   - m → number of rows
   - n → number of columns
   - r → number of rotations
2. Input the matrix elements into a 2D array.
3. Calculate the number of layers:
   - layers = min(m, n) / 2
4. For each layer:
   - Define boundaries:
   - top = layer
   - bottom = m - 1 - layer
   - left = layer
   - right = n - 1 - layer
5. Extract elements of the current layer in order:
   - Top row (left to right)
   - Right column (top+1 to bottom-1)
   - Bottom row (right to left)
   - Left column (bottom-1 to top+1)
   - Store extracted elements in a list.
6. Compute effective rotations:
   - rotations = r % size_of_layer
   - Rotate the list anti-clockwise using:
7. Collections.rotate(list, -rotations)
8. Place the rotated elements back into the matrix in the same order.
9. Repeat for all layers.
10. Print the final rotated matrix.

PROGRAM

```
import java.util.*; public class Solution {
public static void main(String[] args) { Scanner sc = new Scanner(System.in);
int m = sc.nextInt(); int n = sc.nextInt(); int r = sc.nextInt();
```

```
int[][] matrix = new int[m][n];
for (int i = 0; i < m; i++) { for (int j = 0; j < n; j++) {
matrix[i][j] = sc.nextInt();
}
}
int layers = Math.min(m, n) / 2;
for (int layer = 0; layer < layers; layer++) { List<Integer> elements = new ArrayList<>(); int top = layer;
int left = layer;
int bottom = m - 1 - layer; int right = n - 1 - layer;
for (int i = left; i <= right; i++) elements.add(matrix[top][i]);
for (int i = top + 1; i <= bottom - 1; i++) elements.add(matrix[i][right]);
for (int i = right; i >= left; i--) elements.add(matrix[bottom][i]);
for (int i = bottom - 1; i >= top + 1; i--) elements.add(matrix[i][left]);
int size = elements.size(); int rotations = r % size;
Collections.rotate(elements, -rotations);

int index = 0;
for (int i = left; i <= right; i++) matrix[top][i] = elements.get(index++);
for (int i = top + 1; i <= bottom - 1; i++) matrix[i][right] = elements.get(index++);
for (int i = right; i >= left; i--) matrix[bottom][i] = elements.get(index++);
for (int i = bottom - 1; i >= top + 1; i--) matrix[i][left] = elements.get(index++);
}
for (int i = 0; i < m; i++) { for (int j = 0; j < n; j++) {
System.out.print(matrix[i][j] + " ");
}
System.out.println();
}
}
}
```

OUTPUT:

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

- **Sample Test case 0**
- Sample Test case 1
- Sample Test case 2
- Sample Test case 3

Input (stdin)

```
4 4 1
1 2 3 4
5 6 7 8
9 10 11 12
13 14 15 16
```

Your Output (stdout)

```
2 3 4 8
1 7 11 12
5 6 10 16
9 13 14 15
```

RESULT :The program successfully rotates the matrix in the anti-clockwise direction by r times using an efficient layer-by-layer approac